GitHub Code Directory

Computational Companion to
Machine Learning Evolution Equations
(a De Gruyter book)

Joseph Shomberg
Department of Mathematics
Providence College

August 7, 2026

Introduction

This document provides an organized directory of the Python source code developed in support of the numerical experiments presented throughout *Machine Learning Evolution Equations*. The complete collection of programs is maintained in this GitHub repository and serves as the official computational companion to the text.

Herein we describe the purpose of each major program contained in the repository, identify the mathematical model or numerical algorithm implemented by each code, indicate the chapter, section, or figure of the text to which the program is related, and facilitate reproducibility and future extensions of the computational framework.

Throughout the repository, emphasis is placed on producing readable, modular, and well-documented code that closely reflects the mathematical development presented in the accompanying text.

The repository spans the following computational methods and tools:

Time Integration

- Explicit forward Euler
- Backward Euler
- Semi-implicit convex-concave splitting

Machine Learning

- Dataset generation for supervised learning
- GAN and WGAN-GP training
- Physics-informed and residual-based learning

Numerical Simulation

- Newton iteration
- Finite-difference discretizations
- Reaction-diffusion simulations
- Phase-field simulations

Analysis and Visualization

- Statistical evaluation
- Visualization utilities
- Post-processing tools

Dependencies

Typical software requirements include: *Python 3*, *NumPy*, *SciPy*, *Matplotlib*, *TensorFlow* and *Keras*. Additional package requirements are noted where appropriate.

Reproducibility

The repository is intended to promote reproducible computational research. Whenever possible, scripts include comments describing parameter choices, numerical methods, and expected outputs so that readers may reproduce the results appearing in the text.

Code Used for Chapter 3

Scripts are organized by their occurrence in the text.

Forward Euler Approximation for a Nonlinear Ordinary Differential Equation

The following *Python* program implements the forward Euler method used in Example 3.1 and illustrated in Figure 3.1. The script approximates the solution of the nonlinear initial value problem

$$y' = y^2, \qquad y(0) = \frac{1}{1.1},$$

on the interval $[0, 1]$. The exact solution of this differential equation is

$$y(x) = \frac{1}{1.1 - x},$$

which grows rapidly as x approaches 1. This example therefore provides a useful demonstration of how numerical errors accumulate when explicit time-stepping methods are applied to rapidly growing solutions. The figure shows forward Euler approximation compared to the exact solution in order to illustrate the behavior of the numerical method near a singularity.
3.1-Euler_Introduction code

Forward Euler Simulation of the Cancer-Immune System Model

This program implements the forward Euler time discretization used in Example 3.12 and illustrated in Figure 3.2. The script simulates a simplified dynamical system describing the interaction between cancer cells and immune response mechanisms. The model consists of a system of ordinary differential equations governing the temporal evolution of interacting cell populations. Starting from prescribed initial conditions, the solution is advanced in time over the interval $[0, t_{\text{end}}]$ using a uniform time step determined by the total number of steps N. The figure records the computed trajectories and produces graphical visualizations of the population dynamics.
3.2-Euler_Cancer-Immune_I code

Forward Euler Simulation of the Cancer-Immune Interaction Model

The following *Python* program implements the forward Euler time-stepping scheme used in Example 3.13 and illustrated in Figure 3.3. The script numerically simulates a simplified cancer-immune interaction model and performs a parameter sweep over different values of the immune activation parameter A. The system is integrated over the time interval $[0, t_{\text{end}}]$ using a uniform time step determined by the total number of steps N. For each value of A, the forward Euler method produces a numerical trajectory that illustrates the qualitative behavior of the dynamical system, including the possible emergence of steady states or long-time transients. The program records the resulting trajectories and produces plots showing the evolution of the interacting populations over time.
3.3-Euler_Cancer-Immune_II code

Smooth Poisson Initial Data

The following script generates a smooth initial condition on a 128×128 grid by solving a discrete Poisson problem with homogeneous Dirichlet boundary conditions and then applying a small amplitude scaling. The output is shown in Figure 3.4.
3.4-Poisson-smoothIC code

Semi-Implicit Method of the Chafee–Infante Reaction-Diffusion Equation with Smooth Poisson Initial Data

The following *Python* program implements the semi-implicit numerical method described in Section 3.5 and used to produce the solution illustrated in Figure 3.4. The script computes numerical solutions of the two-dimensional Chafee–Infante reaction-diffusion equation

$$u_t - \gamma \Delta u + \kappa \left(u^3 - u \right) = 0$$

on the square domain $[-1, 1] \times [-1, 1]$ subject to homogeneous Dirichlet boundary conditions

$$u = 0 \qquad \text{on } \partial\Omega.$$

Time discretization is performed using a semi-implicit scheme with an Eyre convex-concave splitting. Thus, the linear diffusion operator and the cubic nonlinearity are treated implicitly, while the expansive linear term is treated explicitly. At each time step the resulting nonlinear system is solved on the interior grid by Newton's method. The program generates numerical solution snapshots together with diagnostic quantities such as the quadratic energy, the discrete Lyapunov energy associated with the gradient-flow structure of the equation, and the evolution of the maximum and minimum solution values. These quantities are written to disk for visualization and analysis.
3.5-CIRDE-sim-smooth code

Random Initial Data

The following script generates amplitude-scaled random initial data on a 128×128 grid. Independent values are sampled from the uniform distribution on $[-1, 1]$ at interior grid points, while homogeneous Dirichlet boundary conditions are imposed by setting boundary values to zero. The output is shown in Figure 3.5.
3.6-randomIC code

Semi-Implicit Method for the Chafee–Infante Reaction-Diffusion Equation with Random Initial Data

In the case of the semi-implicit method for the Chafee–Infante equation with random initial data shown in Section 3.5 and Figure 3.5, the numerical simulation is initialized with the following:

$$\phi(0, x) \sim \text{Unif}[-1, 1]$$

independently at each interior grid point, while homogeneous Dirichlet boundary conditions $\phi = 0$ are imposed on $\partial\Omega$.
3.7-CIRDE-sim-random code

Dataset generator for Chafee–Infante Reaction-Diffusion Equation

The following *Python* program generates training and testing data for the inverse Chafee–Infante problem under homogeneous Dirichlet boundary conditions. Each sample is produced in two stages. First, a smooth initial condition is constructed by solving a screened Poisson equation with random forcing. Random initial data can be substituted here. Second, this initial state is evolved forward for a prescribed number of explicit forward Euler steps. The resulting pair

$$(\texttt{src}, \texttt{tar}) = (S_N(u_0), u_0)$$

is then stored in compressed *NumPy* format, where `tar` denotes the initial condition and `src` denotes the corresponding near-equilibrium state after N forward steps. To reduce memory usage, samples are first written in batches and may then be merged into a single compressed dataset file. 3.8-CIRDE-smooth-dataset code

Initial Data for the Spectral Suppression Experiment

Figure 3.7 illustrates the forward evolution of the Chafee–Infante reaction-diffusion equation using a carefully constructed initial condition. The purpose of this experiment is to explicitly demonstrate the mechanism of *spectral suppression* discussed in Chapter 3. In particular, the initial condition is designed to contain both low-frequency and high-frequency spatial modes. More precisely, the initial condition is constructed as a superposition of Dirichlet Laplacian eigenfunctions of varying frequencies. The low-frequency components represent large-scale structure, while the high-frequency components encode fine spatial oscillations. Under forward evolution, the dissipative dynamics of the equation rapidly damp the high-frequency modes, while the low-frequency components persist. As a result, the solution evolves toward a coarse, phase-separated configuration that retains only large-scale features. This experiment therefore provides a direct numerical illustration of the fact that the forward evolution acts as a nonlinear low-pass filter. Information contained in high-frequency modes is irreversibly lost, which is the fundamental source of ill-posedness for the inverse problem studied in this work.

The initial condition used in this experiment is given by the following code:

```
def engineered_sine_ic(M, L, amp_low=initial_amplitude, amp_high=initial_amplitude
                                                    ):
    """
    Construct a spectrally engineered initial condition satisfying
    homogeneous Dirichlet boundary conditions.

    The initial condition is a sum of discrete Dirichlet Laplacian modes:
        phi_{m,n}(x,y) = sin(m pi xi) sin(n pi eta),
    where xi, eta are rescaled coordinates on [0,1].

    We combine one low mode with several higher modes in order to study
    the suppression of high-frequency spectral content under forward
    evolution.

    Parameters
    ----------
    M : int
        Total number of grid points per coordinate direction, including boundary.
    L : float
        Half-width of the domain [-L, L] x [-L, L].
    amp_low : float, optional
        Amplitude of the low-frequency mode.
```

```
    amp_high : float, optional
        Amplitude shared by the high-frequency modes.

    Returns
    -------
    np.ndarray
        Full (M x M) array with zero boundary values.
    """
    x = np.linspace(-L, L, M, dtype=np.float64)
    y = np.linspace(-L, L, M, dtype=np.float64)
    X, Y = np.meshgrid(x, y, indexing="ij")

    # Rescale [-L, L] to [0, 1] so that Dirichlet sine modes vanish on boundary
    XI = (X + L) / (2.0 * L)
    ETA = (Y + L) / (2.0 * L)

    def mode(m, n):
        return np.sin(m * np.pi * XI) * np.sin(n * np.pi * ETA)

    u0 = (
        amp_low * mode(2, 2)
        + amp_high * mode(18, 18)
        + amp_high * mode(24, 20)
        + amp_high * mode(20, 24)
    )

    # Enforce boundary values exactly
    u0[0, :] = 0.0
    u0[-1, :] = 0.0
    u0[:, 0] = 0.0
    u0[:, -1] = 0.0

    return u0.astype(np.float32)
```

3.9-CIRDE-sim-suppress code

Code Used for Chapter 4

Physics-Informed Training Chafee–Infante with Smooth Poisson Initial Data

We now record a representative training script for the inverse learning problem associated with the Chafee–Infante equation using the smooth initial data described in Chapter 3. To train the same generator and critic architecture on random initial data, one simply replaces the smooth-data training and testing datasets with the corresponding random-data datasets; *no further structural change to the script is required.* In this setting, the input to the network is a near-equilibrium state obtained after a fixed number of forward time steps, and the target output is the corresponding initial condition.

The model implemented is a Wasserstein generative adversarial network with gradient penalty (WGAN-GP). The generator is a U-Net style encoder-decoder with skip connections, while the critic is a PatchGAN type convolutional network. In addition to the adversarial objective, the training loss includes several physically and statistically meaningful terms: a Lyapunov energy mismatch, a residual loss obtained by forward evolution of the predicted initial condition, a mean absolute error (MAE) term, mean and variance penalties, and a gradient-based regularization term.
4.1-CIRDE-WGANGP-smooth code

Evaluation of the Learned Inverse Solution

We now record a unified evaluation script for the inverse Wasserstein GAN model trained on smooth Chafee–Infante data. The purpose of this script is fourfold. First, it produces qualitative plots comparing observed near-equilibrium states, generated initial conditions, and true initial conditions. Second, it computes full-test reconstruction statistics, including the mean and standard deviation of the mean absolute error (MAE) and the physics-based residual. Third, it estimates empirical probabilities associated with the accuracy and physical consistency of the learned inverse map. Finally, it reports whether the trained model satisfies the probabilistic notion of machine learned inverse solution introduced in Chapter 4.
4.2-CIRDE-WGANGP-smooth-testing code

Pixel-Wise Training Chafee–Infante with Smooth Poisson Initial Data

This script performs an implementation of the MAE-only training configuration used to study the effect of pixel-wise reconstruction loss in the inverse problem for the Chafee–Infante reaction-diffusion equation to accompany Section 4.8.2. The architecture and data pipeline are identical to those used in the full WGAN-GP framework described in Section . The essential difference lies in the loss design: all physics-informed terms are removed, and the generator is trained using only the mean absolute error loss. A small adversarial component is retained to stabilize training, but it does not enforce dynamical consistency.

This configuration isolates the statistical behavior of the reconstruction loss and allows direct comparison with the physics-informed models developed later.
Summary of configuration:

- Input: terminal states u_T
- Output: reconstructed initial conditions $\tilde{u}_0$
- Loss: $\mathcal{L} = \lambda_{\text{MAE}} \|u_0 - \tilde{u}_0\|_1 + \lambda_{\text{adv}} \mathcal{L}_{\text{WGAN}}$
- Physics terms: *not included*

4.3-CIRDE-WGANGP-MAE-only code

Reports and ML/Probabilistic Solution Metrics

This script is used to generate a detailed report on trained generator models. Of particular importance are the metrics used in the probabilistic notion of a machine-learned solution to PDE inversion problems. These metrics are determined by evaluating the trained generator on the yet unseen testing dataset.
4.8-CIRDE-reports code

Code Used for Chapter 5

Allen–Cahn Simulation

This script provides a reference implementation of a semi-implicit convex splitting scheme for the two-dimensional Allen–Cahn equation with periodic boundary conditions. The code is used to generate the datasets and figures presented in Chapter 5. The numerical method treats the Laplacian implicitly and the nonlinear term explicitly, yielding an unconditionally stable discretization that

preserves the dissipative structure of the equation. In addition to the solution snapshots, the code computes diagnostic quantities including the Lyapunov energy, an approximate interface measure, and the evolution of the maximum and minimum values.
5.4-AC-simulation code

Ternary Cahn–Hilliard Simulation

The following code implements the explicit ternary Cahn–Hilliard simulation with periodic boundary conditions used to generate the phase-separation figures in Section 5.6.8 and Figures 5.4 and 5.5. The purpose of the script is not to provide a high-order production solver, but rather a transparent and reproducible computational laboratory for dataset generation. The method uses periodic finite differences, forward Euler time stepping, projected chemical potentials, and a mass-corrected projection onto the Gibbs simplex after each update. This final projection enforces the physical constraints

$$0 \le c_i \le 1, \qquad c_1 + c_2 + c_3 = 1,$$

while keeping the component masses close to their prescribed initial values. Thus, the method should be understood as

$$\text{forward Euler} + \text{mass-corrected simplex projection},$$

rather than as the unmodified forward Euler discretization. This distinction is important: the unprojected Cahn–Hilliard flow conserves mass under periodic boundary conditions, but explicit Euler steps are not *positivity preserving*. For machine-learning datasets, the projected scheme produces physically meaningful concentration fields while retaining the qualitative coarsening and phase-separation dynamics.
5.6.1-tCHE-simulation code

Physics-Informed Training Ternary Cahn–Hilliard with Eyre-Noisy Initial Data

The following is *Python* code for training a physics-informed reconstruction model on the ternary Cahn–Hilliard equation with Eyre-noisy initial data. This inversion problem appears in Section 5.6.15.

The MAE-only experiment is contained in Example 5.49 and Figures 5.9 and 5.10.

The MAE and physics-informed results appear in Example 5.51 and Figures 5.11 and 5.12.
5.6.2-tCHE-full-Eyer code

Evaluation of the Learned Inverse Solution

This is the companion model testing script for the training code developed for the physics-informed ternary Cahn–Hilliard inversion problem mentioned above. Here we evaluate the reconstruction model over the entire testing dataset and perform analyses of all relevant errors: purity loss, simplex loss, residual loss, energy loss, mass loss, pixel-wise loss, and statistical loss.
5.6.X-Evaluation-tCHE-models code

Viscous Burgers Simulation

The following code generates the forward simulation used in Section 5.7 and Figure 5.13. We solve the viscous Burgers equation

$$u_t + uu_x = \nu u_{xx}$$

on a periodic one-dimensional domain. The method uses operator splitting. The nonlinear transport equation

$$u_t + \left(\frac{u^2}{2}\right)_x = 0$$

is advanced using a conservative local Lax–Friedrichs flux, while the diffusion equation

$$u_t = \nu u_{xx}$$

is advanced exactly in Fourier space. This separates the two essential mechanisms of the equation, nonlinear steepening and viscous smoothing.
5.7-VBE-simulation code

Two-Dimensional Navier–Stokes Forward Simulation

The following code implements a projection method for the two-dimensional incompressible Navier–Stokes equations on a periodic domain. The results appear in Section 5.9 and Figure 5.14. An intermediate velocity field is first computed by advancing the nonlinear advection and viscous diffusion terms. This intermediate field is then projected onto the space of divergence-free vector fields by solving a periodic pressure Poisson equation in Fourier space. The method follows the structure described in the main text:

$$u^* = u^n + \Delta t \left[-\left(u^n \cdot \nabla\right) u^n + \nu \Delta u^n \right],$$

followed by

$$\Delta p^{n+1} = \frac{1}{\Delta t} \nabla \cdot u^*,$$

and

$$u^{n+1} = u^* - \Delta t \nabla p^{n+1}.$$

5.9-NSE-simulation code

Kuramoto–Sivashinsky Simulation

The following code implements the forward solver used to simulate the Kuramoto–Sivashinsky equation in Section 5.10 and Figure 5.15. The method is spectral in space and semi-implicit in time, allowing the stiff fourth-order dissipation to be treated stably while retaining an explicit nonlinear transport update.
5.10-KSE-simulation code

Nonlinear Schrödinger Simulation

The following code generates the forward simulation used in Section 5.13 and Figure 5.16. We solve the one-dimensional nonlinear Schrödinger equation

$$iu_t + u_{xx} + \kappa |u|^2 u = 0$$

on a periodic interval. The method is a split-step Fourier method. The dispersive linear part is solved exactly in Fourier space, while the nonlinear phase rotation is solved exactly in physical space. This makes the method especially well suited for illustrating wave-packet propagation, interference, and phase-sensitive dynamics.
5.13-NLS-simulation code

Thin-Film Simulation

The thin-film equation describes the evolution of a viscous liquid film under the action of surface tension. Surface-tension forces reduce curvature, causing small-scale features to smooth while conserving the total film volume. The simulation presented here illustrates the qualitative dynamics of this fourth-order nonlinear diffusion process. The following code generates the thin-film simulation used in Section 5.14 and Figure 5.17. We solve the periodic thin-film equation

$$u_t + \nabla \cdot (u^n \nabla \Delta u) = 0$$

on a two-dimensional periodic domain. A finite-difference discretization with periodic boundary conditions is employed. The nonlinear mobility u^n is evaluated explicitly, and the equation is advanced using a sufficiently small time step for stability. Beginning from a slightly perturbed uniform film, the solution evolves toward smoother configurations while conserving mass, illustrating the characteristic behavior of capillary-driven thin-film flow.
5.14-THIN-simulation code